

Safe Serverless Workflow Orchestration through Typestate Verification, Retry Safety, and Compensation Checking

Anonymous Author(s)

Anonymous Institution
anonymous@example.org

Abstract. Serverless workflow orchestration platforms such as AWS Step Functions coordinate fleets of functions and managed services, but their workflows are written as untyped JSON state machines (the Amazon States Language, ASL) whose correctness is checked only at run time. Incompatible task interfaces, out-of-order business transitions, unsafe retries of non-idempotent operations, and missing compensations therefore surface as production incidents rather than build errors. We present STEPCHECK, a static verifier that brings four complementary checks—data-contract compatibility, typestate (business-protocol) conformance, retry safety, and Saga compensation completeness—to workflows *before* deployment, and compiles verified workflows back to executable ASL. Because real ASL omits the semantics these checks need (task types, idempotency, compensation), STEPCHECK pairs sound native analyses with a lightweight annotation layer whose defaults are *inferred* from task names and resource bindings, so existing workflows can be analysed with zero up-front effort and refined incrementally. STEPCHECK is implemented in Rust as a modular frontend/pass pipeline (2.6k LOC) and evaluated on a corpus of **193** real Step Functions workflows mined from public AWS repositories. The native checks flag genuine defects in the wild; a mutation study injecting 429 defects of four classes is detected with 100% recall; name-based inference reaches 83–91% accuracy against a hand-labelled gold set; verification costs 19 μ s per workflow on average; a verified workflow deploys and executes unmodified on AWS Step Functions with zero runtime overhead; and the architecture generalizes to a second workflow language (CNCf Serverless Workflow) with no change to the analyses.

Keywords: Serverless computing · Workflow orchestration · AWS Step Functions · Static verification · Typestate · Saga transactions · Domain-specific languages.

1 Introduction

Cloud-native applications increasingly decompose business logic into independently deployable functions that are coordinated by managed *workflow orchestration* services. Among these, AWS Step Functions has become one of the most

widely adopted platforms in industry [13,23,16]. A Step Functions workflow is a state machine written in the *Amazon States Language* (ASL), a JSON configuration format that describes task invocations, branching, parallelism, retries, and error handling, and that executes on top of AWS Lambda and dozens of service integrations [2].

While orchestration platforms simplify *running* distributed workflows, they provide little support for *reasoning about workflow correctness before deployment*. An ASL state machine is an untyped artifact: the platform validates its JSON schema and the existence of transitions, but not whether the data produced by one task matches what the next consumes, whether tasks are invoked in a valid business order, whether an automatically retried task is safe to repeat, or whether a long-running transaction can roll back the effects it has already committed. As a consequence, four recurring classes of defect surface only at run time:

Contract mismatches. A task reads a field its predecessor never produces (e.g. `ChargeCard` expects `total` but receives `amount`).

Protocol (typestate) violations. An edit reorders the machine so a task runs out of business order (shipping before payment)—valid JSON, invalid protocol.

Unsafe retries. A retry policy re-invokes a non-idempotent operation after a partial success, duplicating its effect [18].

Missing compensation. A persistent operation reserves a resource the workflow cannot release when a later step fails—an incomplete Saga [14].

A central obstacle to checking these properties is that ASL *omits the very information the checks require*. ASL has no notion of a task’s input or output type, no business-state labels, no idempotency annotation, and no concept of “compensation” beyond a generic `Catch`. Existing language-level approaches sidestep this by introducing a new programming model with a proprietary runtime [6,25,21]; but the millions of workflows already written in ASL cannot benefit, and re-platforming is rarely an option.

Approach. We present STEP CHECK, a static verifier that brings four complementary checks to serverless workflows *before deployment* while remaining fully compatible with AWS Step Functions. STEP CHECK is organised around two ideas. First, a *two-layer* analysis model: a set of *native* checks operate soundly on raw ASL with no extra input (structural well-formedness and ASL-semantic data-binding checks), while a set of *semantic* checks (typestate, retry safety, compensation) draw on a lightweight *annotation* layer. Second, because most existing workflows carry no annotations, the missing semantics are *inferred* from task names and resource bindings by a documented, configurable heuristic, giving a zero-effort default analysis that developers can override or refine. Verified workflows are compiled back to executable ASL, so STEP CHECK introduces no runtime component and no lock-in. A typed builder frontend additionally lets greenfield workflows be authored with full static guarantees and the same compilation target.

Contributions.

- A practical model and four static analyses —contract/data-flow, typestate, retry safety, compensation completeness— for serverless workflows, framed operationally rather than through heavy formalism (Sections 3–4).
- A two-layer design that verifies *existing* ASL workflows by pairing sound native checks with an annotation layer whose defaults are inferred from naming/resource conventions (Section 3).
- STEP CHECK, an open, modular implementation in Rust (2.6k LOC) with pluggable frontends (raw ASL, a typed DSL, and the CNCF Serverless Workflow DSL) and analysis passes, a diagnostics engine, an ASL emitter, and a mutation-based error-injection tool (Section 5).
- An evaluation on **193 real** Step Functions workflows mined from public AWS repositories: in-the-wild findings, a 429-mutant detection study (100% recall), inference accuracy against a hand-labelled gold set (83–91%), sub-millisecond verification cost, and a real AWS round-trip (Section 6).

Research questions. **RQ1:** Can a static verifier detect the four defect classes in serverless workflows before deployment? **RQ2:** Can the semantics ASL omits be recovered cheaply enough to analyse existing workflows? **RQ3:** What is the cost of verification and what runtime overhead does it impose? We answer these in Section 6.

2 Background and Motivation

2.1 Step Functions and the Amazon States Language

A Step Functions workflow is a state machine: a **StartAt** entry and a map of named *states*. A **Task** state invokes a Lambda function or a service integration; **Choice** branches; **Parallel** and **Map** fan out; **Wait**, **Pass**, **Succeed**, and **Fail** complete the set. Data flows through a JSON document that each state reshapes with JSONPath fields (**Parameters**, **InputPath**, **ResultPath**, **OutputPath**), where a key suffixed **.\$** denotes a path reference. States carry retry (**Retry**) and error-handling (**Catch**) policies. Listing 1.1 shows a representative task drawn verbatim from our corpus.

Listing 1.1. A real ASL task state (excerpt, **ship-order**).

```
"Get Customer Status": {
  "Type": "Task",
  "Resource": "arn:aws:states:::lambda:invoke",
  "OutputPath": "$.Payload",
  "Parameters": { "FunctionName": "${LambdaGetCustomerStatus}",
                  "Payload.$": "$.detail.customer" },
  "Retry": [ { "ErrorEquals": ["Lambda.ServiceException"],
              "IntervalSeconds": 2, "MaxAttempts": 6, "BackoffRate": 2
            } ],
  "Next": "Do Fraud Check"
}
```

The platform validates such a definition’s JSON schema and that every transition target exists, then executes it. It does *not* reason about whether `$.detail.customer` is present, whether retrying this task is safe, or how the workflow recovers if a later step fails.

2.2 Four classes of latent defect

Consider the canonical order-processing workflow

`CreateOrder` $\rightarrow$ `ReserveInventory` $\rightarrow$ `ChargeCard` $\rightarrow$ `ShipOrder`,

which we use as a running example. It exhibits all four defect classes when edited carelessly. (1) *Contract*: if `ReserveInventory` produces `{orderId, amount}` but `ChargeCard` consumes `total`, the payment task fails at run time on a missing field. (2) *Typestate*: reordering to `ShipOrder` $\rightarrow$ `ChargeCard` ships before payment—valid JSON, invalid business protocol. (3) *Retry*: `ChargeCard` is non-idempotent; a broad retry (`ErrorEquals:[States.ALL]`) may charge twice. (4) *Compensation*: `ReserveInventory` commits a reservation; if `ChargeCard` fails with no `ReleaseInventory` path, the reservation leaks. None of these is caught by the platform; all are common in practice [18,16].

2.3 The information gap

The obstacle to checking these properties statically is not the analysis but the *input*. Raw ASL encodes control flow and JSONPath data plumbing, but it does not record: the *type* of data a task consumes or produces; the *business state* a task represents; whether a task is *idempotent*; or whether a task is *persistent* and which action *compensates* it. A verifier for existing workflows must therefore either (a) restrict itself to properties decidable from ASL alone, or (b) recover the missing semantics. STEP CHECK does both: it runs sound native checks that need nothing beyond ASL, and it recovers the remaining semantics through a lightweight annotation layer whose defaults are inferred from the strong naming and resource conventions that pervade real workflows (Section 3). Figure 1 contrasts the resulting development process with the status quo.

3 Approach

3.1 A two-layer analysis model

STEP CHECK separates what can be checked from ASL alone from what needs recovered semantics. *Native* analyses (Layer A) are sound and require no annotations: structural well-formedness, and ASL-semantic data-binding checks such as a `waitForTaskToken` task that never passes its task token, misuse of the `$$Map` context, and syntactically invalid JSONPath payloads. *Semantic* analyses (Layer B) —typestate, retry safety, compensation, and typed contract compatibility— consume an *annotation* layer that records the task semantics ASL

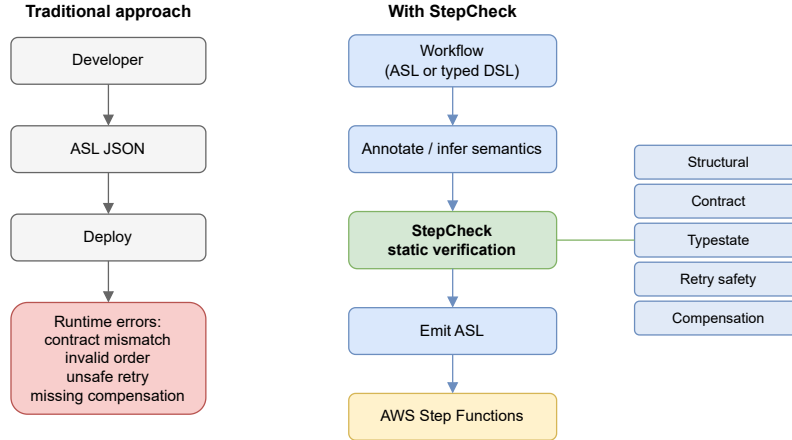


Fig. 1. Status quo (left): ASL is deployed directly and defects surface at run time. STEPCheck (right): semantics are recovered by annotation or inference, five analyses run before deployment, and only verified workflows are compiled to ASL.

omits. The annotation layer has two sources, in priority order: explicit annotations (authored by hand or produced by the typed DSL frontend), and *inference* that supplies defaults from naming and resource conventions. Findings derived from inferred facts are reported at *warning* severity, while findings derived from declared facts are *errors*; this single rule keeps confidence visible throughout the tool’s output.

3.2 Workflow model

We model a workflow as a graph of states with a designated start. A *task* is characterised, beyond its control-flow successors, by four pieces of semantic information used by the checks:

Contract. A task consumes an input record and produces an output record.

We write a task as $T : A \rightarrow B$, where A and B name record types (sets of fields). Two adjacent tasks $T_1 : A \rightarrow B$ and $T_2 : C \rightarrow D$ *compose* only if B provides every field C requires. Natively (without declared types) the contract layer instead checks that JSONPath references resolve to data the workflow actually binds.

Typestate. Each task also advances a *business state*: T moves the workflow from state s_{in} to s_{out} . A workflow is well-ordered when each task consumes the state its predecessor produced. An optional *protocol* lists the permitted state transitions (e.g. *created* $\rightarrow$ *reserved* $\rightarrow$ *paid* $\rightarrow$ *shipped*), so that a task performing an undeclared transition is rejected.

Idempotency. A task is *idempotent* if repeating it causes no additional external effect. Idempotent tasks may be retried freely; non-idempotent tasks may not be retried on broad error classes.

Persistence and compensation. A task is *persistent* if it commits a durable resource (a reservation, a charge, a created record) that a later failure should undo. A persistent task needs a *compensation*: an explicit handler or a **Catch** path to a compensating state.

This vocabulary is deliberately lightweight—there is no separate formal calculus to maintain—yet it is exactly what the four checks of Section 4 consume. We illustrate it with the order workflow throughout.

3.3 The annotation layer

Explicit semantics are supplied either through a *sidecar* file attached to an existing ASL workflow (Listing 1.2) or, for greenfield workflows, through a typed builder DSL that emits both the ASL and its annotations (Listing 1.3). The two routes resolve to the same internal representation, so the analyses are agnostic to how a workflow was authored.

Listing 1.2. A sidecar annotation (TOML), excerpt.

```
[schemas.OrderReserved]
fields = ["orderId", "amount", "reservationId"]
[tasks.ChargeCard]
idempotent = false
persistent = true
compensation = "RefundPayment"
state_in = "OrderReserved"
state_out = "OrderPaid"
```

Listing 1.3. The order workflow in the typed DSL (excerpt).

```
Builder::new("order-processing", "CreateOrder")
  .schema("OrderReserved", &["orderId", "amount", "reservationId"])
  .protocol("OrderReserved", "OrderPaid")
  .task("ChargeCard", "OrderReserved", "OrderPaid",
    /*idempotent*/ false, /*persistent*/ true,
    /*compensation*/ Some("RefundPayment"),
    /*next*/ Some("ShipOrder"), /*catch*/ Some("RefundPayment"))
```

3.4 Inference of default annotations

Most existing workflows carry no annotations, yet their task names and resource ARNs are highly conventional. STEP CHECK exploits this with a documented, configurable heuristic that classifies each task from a lowercased *signal* (its name, its **Resource**, and parameters such as **FunctionName**). The heuristic uses four small keyword sets: *read* verbs (`get`, `list`, `describe`, `query`, `validate`,

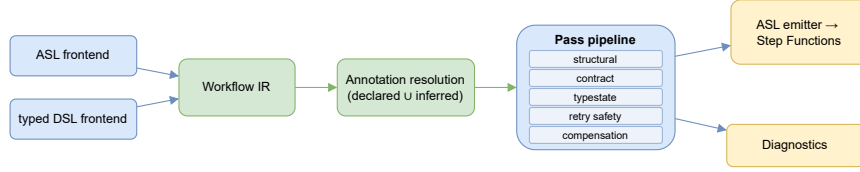


Fig. 2. STEPCHECK architecture: frontends lower to a common IR; annotation resolution recovers semantics; a pipeline of passes verifies; an emitter targets Step Functions.

...) imply idempotent and non-persistent; *write* verbs (**create**, **charge**, **send**, ...) imply non-idempotent; *resource-acquiring* verbs (**reserve**, **pay**, **provision**, **putItem**, ...) additionally imply persistent; and *notification* verbs/resources (**notify**, **publish**, **:sns:**, ...) are non-idempotent but, being fire-and-forget, never persistent. When no keyword matches, the heuristic *abstains*, leaving the task unclassified rather than guessing. Inference thus yields a zero-effort default whose mistakes (Section 6) are visible (warnings) and correctable (an explicit annotation always wins).

3.5 Architecture

Figure 2 shows the pipeline. A *frontend* lowers a surface syntax to a format-agnostic intermediate representation (IR); annotation resolution attaches declared and inferred semantics to the IR; a *pass* pipeline runs the five analyses, each emitting coded diagnostics; and an *emitter* lowers verified workflows back to ASL. The design is modular along two axes: a new input format is a new **Frontend**, and a new check is a new **Pass** (Section 5).

4 Static Analyses

STEPCHECK runs five passes over the IR. Each pass is independent and emits diagnostics with a stable code (Table 1), a severity, the offending state, and a remediation note. We describe each pass operationally; all are linear in the number of states and run over nested **Map/Parallel** sub-machines.

Structural validation (SC0xxx). A reachability traversal from **StartAt** (following normal *and* error transitions) checks that all transition targets exist, that no state is unreachable, that every non-terminal offers a continuation, that a terminal is reachable, that **Choice** states declare a **Default**, and that **Map/Parallel** states are well-formed—sound checks that catch defects AWS reports only at run time.

Contract / data-flow (SC1xxx). Natively, the pass mines every JSONPath reference (**.\$** payloads and **Choice** variables) and checks ASL-semantic bindings: a **waitForTaskToken** task must pass **\$\$Task.Token** or its callback can never be delivered (SC1001); the **\$\$Map** context is valid only inside a **Map** (SC1002);

Table 1. Diagnostic codes emitted by STEP CHECK. Severity † is *error* when derived from declared facts and *warning* when derived from inference.

Code	Pass	Meaning
SC0001–0010	structural	undefined start/transition target, unreachable state, dead-end, non-exhaustive Choice , empty Map/Parallel , malformed state
SC1001–1003	contract	waitForTaskToken without a task token, \$\$. Map misuse, invalid JSONPath payload
SC1010	contract	typed contract mismatch: successor requires a field the predecessor does not produce
SC2001–2002	typestate	out-of-order composition; undeclared business-protocol transition
SC3001†	retry	non-idempotent task retried on a broad error class
SC4001–4002†	compensation	persistent task with no compensation/handling; compensation names a missing state

and every payload must be a valid path or **States.*** intrinsic (SC1003). With declared schemas it also performs typed checking (SC1010): following control flow through transparent states, each successor’s required fields must be a subset of the producer’s output—the **amount/total** defect of Section 2.2.

Typestate (SC2xxx). Using **state_in/state_out** labels, the pass checks that adjacent tasks compose (predecessor output state = successor input state, SC2001), catching re-orderings such as ship-before-pay, and that each task’s transition is a permitted protocol edge when one is declared (SC2002). Absent annotations it is a no-op.

Retry safety (SC3xxx). A task with a **Retry** policy whose **ErrorEquals** includes a broad class (**States.ALL/States.TaskFailed**) is flagged SC3001 if its idempotency resolves to *non-idempotent*. Retries restricted to transient errors (throttling, **ServiceException**) are not flagged, as they fire before any effect.

Compensation completeness (SC4xxx). A *persistent* task needs either a declared compensation or a **Catch** path; one with neither leaves its effect uncompensated (SC4001), and a compensation naming a non-existent state is SC4002.

Worked example. Running STEP CHECK on the well-formed order workflow reports no findings. Reordering it to ship before charging (Section 2.2) yields five errors with no false positives: two SC2001 typestate mismatches (**ShipOrder** consumes *OrderPaid* after **ReserveInventory** produced *OrderReserved*; and **ChargeCard** consumes *OrderReserved* after **ShipOrder** produced *OrderShipped*) and three SC1010 contract mismatches (**paymentId**, **amount**, and **reservationId** are required but not produced upstream). The two checks are complementary: typestate explains the ordering fault, the contract check pinpoints the data that breaks.

5 Implementation

STEPCHECK is implemented in Rust as a single command-line tool, `stepcheck` (2,359 lines; Table 2). We chose Rust deliberately: a standalone analyzer with a `serde`-based parser is what makes it possible to ingest the *existing* ASL JSON found in real repositories—rather than only workflows authored in a new embedded language—and it yields a single dependency-light binary with predictable, sub-millisecond performance.

5.1 A modular frontend/pass pipeline

The architecture’s extensibility rests on two traits (Listing 1.4). A `Frontend` lowers a surface syntax to the workflow IR; a `Pass` reads only the IR and writes diagnostics. Adding an input format is a new `Frontend`; adding a check is a new `Pass`; everything else is unchanged. We exercised this directly: the tool ships three frontends—the typed DSL, raw ASL, and the CNCF Serverless Workflow DSL [9]—and the last was added *without changing the IR or any analysis pass* (Section 6.7), turning the extensibility claim into a demonstrated property.

Listing 1.4. The two extension points.

```
pub trait Frontend { fn load(&self, src: &str) -> Result<Workflow>; }

pub trait Pass {
  fn id(&self) -> &'static str;
  fn run(&self, wf: &Workflow, sink: &mut DiagnosticSink);
}
```

5.2 Components

Workflow IR. A format-agnostic graph of states (`Task`, `Choice`, `Parallel`, `Map`, `Wait`, `Pass`, `Succeed`, `Fail`) with control flow, JSONPath data-flow fields, retry/catch policies, nested sub-machines, and a slot for resolved annotations.

ASL frontend. To tolerate the diversity of real ASL—intrinsic functions, `.$` parameters, the `$$` context object, SAM `${}` placeholders, classic `Iterator` vs. distributed `ItemProcessor`, and recent JSONata features—the frontend lowers from a generic `serde_json` value rather than rigid structs, so unknown fields never cause a hard failure. It parses all 193 corpus workflows without error.

Annotation resolution and inference. Loads a TOML sidecar, resolves schema names to field sets, and fills the remaining gaps with the naming/resource heuristic of Section 3.4.

Passes and diagnostics. The five analyses of Section 4, plus a diagnostics engine that renders findings for a terminal or as JSON.

Table 2. Implementation size (lines of Rust).

Component	LOC
Workflow IR	277
Frontends (ASL parse/emit, typed DSL, CNCF Serverless Workflow)	664
Annotations + inference	251
Analysis passes (5 + manager)	650
Diagnostics	115
Mutation engine	184
CLI + statistics + evaluation harness	490
Total	2,631

ASL emitter and round-trip. Lowers a verified IR back to ASL JSON for deployment. The emitter is the DSL frontend’s code-generator and the basis of the AWS round-trip (Section 6); deploying its output to AWS surfaced—and we fixed—a real bug where an explicit **End** was emitted on **Succeed**/**Fail** states, which Step Functions rejects.

CLI and error injection. **stepcheck** exposes **check**, **infer**, **stats**, **eval**, **emit**, and **mutate**. The **mutate** command injects exactly one defect of a chosen class into a workflow (a broken JSONPath binding, a broad retry on a non-idempotent task, a removed compensation, or a dangling transition) and re-emits deployable ASL; it drives the mutation study of Section 6.

6 Evaluation

We evaluate STEP CHECK on real serverless workflows, asking: can it detect the four defect classes before deployment (RQ1); can it recover the semantics ASL omits cheaply enough to analyse existing workflows (RQ2); and what does verification cost, at build and run time (RQ3)? Every number below is produced by the released tool over a committed corpus and is reproducible with the commands shown.

6.1 Setup and corpus (RQ1)

We mined **193** Step Functions workflows from two public AWS repositories, `aws-samples/aws-stepfunctions-examples` and `aws-samples/step-functions-workflows-collection`, taking every file that parses as an ASL state machine. The corpus (Fig. 3) is realistic and diverse: 1,355 states in total, a median of 6 states per workflow (max 33; Fig. 3a), and broad use of **Choice** (166), **Map** (64), **Parallel** (33; Fig. 3b), retries (84 workflows), and catch handlers (51 workflows). All experiments ran on a commodity laptop with a release build of STEP CHECK (Rust 1.96).

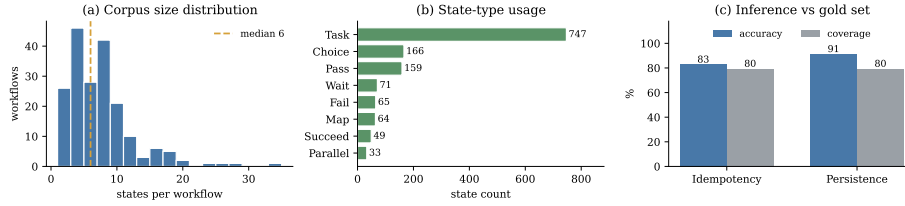


Fig. 3. Evaluation overview. (a) Distribution of workflow sizes and (b) state-type usage across the 193-workflow corpus; (c) inference accuracy and coverage for idempotency and persistence against the hand-labelled gold set (Section 6.4).

Table 3. In-the-wild findings over 193 unmodified workflows (`stepcheck eval corpus/asl -infer`).

Tier	Code	Count	Finding
Sound (native)	SC0007	9	Choice without Default
	SC0010	1	malformed state (misplaced directive)
Inferred (triage)	SC3001	36	broad retry on (inferred) non-idempotent task
	SC4001	117	(inferred) persistent task without compensation

6.2 Findings in the wild (RQ1)

Run unmodified over the corpus (with inference enabled), STEPCheck flags **86 of 193** workflows. We separate *sound native* findings, which need no annotations and which we manually confirmed, from *inference-driven* findings, which are warnings requiring triage (Table 3). Among the native findings, nine workflows contain a **Choice** with no **Default** (SC0007)—a recognised cause of **States.NoChoiceMatched** run-time failures—and one is genuinely malformed (SC0010): an `aws-samples` example nests a top-level **QueryLanguage** directive *inside* its **States** block, where Step Functions would reject it. The inference-driven warnings (SC3001: 36; SC4001: 117) mark non-idempotent tasks retried broadly and persistent tasks lacking a compensation path; as Section 6.4 shows, these are high-recall but include false positives (e.g. a **RefundPayment** compensator inferred as needing its own compensation), which is precisely why they are surfaced as warnings rather than errors.

6.3 Mutation-based detection (RQ1)

Because correct real workflows expose few of the deeper defects, we measure detection power by *error injection*. For each defect class we inject exactly one defect per applicable workflow with `stepcheck mutate` and re-verify; a defect counts as *detected* only if the expected diagnostic’s count strictly increases over the unmutated baseline (so we measure detection of the *injected* defect, not pre-existing findings). Across **429** mutants spanning the four classes, STEPCheck

Table 4. Mutation study: injected defects detected (delta over baseline).

Defect class	Check	Applicable	Detected	Recall
Broken data binding	SC1003	141	141	100%
Unsafe retry	SC3001	103	103	100%
Missing compensation	SC4001	19	19	100%
Dangling transition	SC0002	166	166	100%
Total		429	429	100%

Table 5. Inference accuracy vs. a hand-labelled gold set (44 tasks, 10 workflows).

Property	Labelled	Predicted	Abstained	Correct	Accuracy	Coverage
Idempotency	44	35	9	29	82.9%	79.5%
Persistence	44	35	9	32	91.4%	79.5%

achieves **100% recall** (Table 4), and it never reports the expected defect on the corresponding originals. We stress that this measures *reliability of each check on the defect class it targets*, not detection of arbitrary bugs; the mutations are constructed to fall within each check’s remit.

6.4 Inference accuracy (RQ2)

The semantic checks depend on recovering idempotency and persistence. We hand-labelled a gold standard of **44 tasks** across 10 workflows (spanning a Saga, an inventory domain, account vending, data quality, and text processing), labelling each task by its true operational semantics, and compared the inference to it (Table 5). The heuristic abstains on $\sim 20\%$ of tasks (unconventional verbs such as *confirm*, *audit*, *aggregate*); on the tasks it does classify it reaches **82.9%** accuracy for idempotency and **91.4%** for persistence (Fig. 3c). Its errors are instructive and bounded: *cancel* operations (idempotent in effect) are flagged non-idempotent, and a *refund* compensator is flagged persistent. Because inference only supplies *defaults*, a one-line annotation corrects any such case, and all inferred findings are warnings. This answers RQ2: useful semantics are recovered at zero authoring cost, with residual error made visible and cheaply correctable.

6.5 Verification cost and runtime overhead (RQ3)

Verification is cheap. Averaged over the corpus, the five passes take **18.7 μ s** per workflow (median 14.9 μ s, max 111.8 μ s); the entire 193-workflow corpus verifies in **3.6 ms**. Because all analysis happens before deployment and the output is ordinary ASL, the *runtime* overhead introduced by STEP CHECK is **zero**: the generated workflow executes on the standard Step Functions runtime with no added middleware, sidecar, or instrumentation. This distinguishes STEP CHECK from runtime approaches to fault tolerance and transactions [25,6].

6.6 Conciseness and AWS round-trip

The typed DSL is also more concise than the ASL it generates: the order workflow is 20 declarative DSL statements that expand to a 121-line ASL definition while additionally encoding five record schemas, a four-edge protocol, and per-task idempotency, persistence, and compensation—semantics ASL cannot express.

Finally, to confirm the compatibility claim end-to-end, we deployed the DSL-verified order workflow to real AWS Step Functions. The emitted ASL was deployed *unmodified* (only binding `FunctionName` to a deployed Lambda ARN) as an Express state machine and executed synchronously; the execution reached **SUCCEEDED** with the input propagating through all four task steps to the terminal state, billed at 500 ms / 64 MB. The temporary resources were then removed. A verified **STEPCHECK** workflow thus runs on the unmodified industrial platform.

6.7 Cross-format generalization: CNCF Serverless Workflow

To test that the approach generalizes beyond AWS, we added a third frontend for the vendor-neutral CNCF Serverless Workflow DSL [9] and ran it on **66** example workflows from the specification’s repository. The frontend lowers the DSL’s tasks (`call/do/fork/for/switch/ try/listen/...`) to the *same IR without changing the IR or any analysis pass*, and parses all 66 workflows with no failures. The structural, typestate, retry-safety, and compensation analyses apply unchanged; only the ASL-specific data-binding checks (SC1001–SC1003) stay silent—correctly, since the CNCF DSL uses runtime expressions rather than JSONPath. A mutation study on this corpus detects every injected defect for which the class has an injection site (retry 4/4, structural 9/9); contract- and compensation-class mutations have no sites in these mostly single-task feature demos.

Crucially, the CNCF DSL can declare JSON Schemas for task input and output. Where it does, the typed contract check (SC1010) runs at *full precision with no inference*: a reordered, schema-annotated order workflow is rejected with three SC1010 errors derived directly from the declared schemas (e.g. `shipOrder` requires `paymentId`, which its new predecessor does not produce)—the same defect the inference layer only approximates on raw ASL, here established exactly. The asymmetry is informative: CNCF makes data contracts explicit, closing that gap natively, but—like ASL—leaves idempotency and compensation implicit, so those checks still rely on annotation or inference in both ecosystems.

6.8 Threats to validity

Construct. The mutation study reports 100% recall by design: each mutation targets a defect within a check’s remit, so the result attests to the reliability of each check, not to coverage of all possible bugs. We report it alongside the in-the-wild findings, which are not constructed. *Internal.* Gold labels for idempotency/persistence are judgements; we documented our labelling criteria and release the labels for scrutiny, and the contested cases (cancel, refund) are

exactly where the paper claims inference is imperfect. *External.* The corpus is drawn from AWS sample repositories, which may over-represent didactic workflows; we mitigate this by taking *all* parseable workflows rather than a curated subset, yielding a realistic spread of sizes and features. *Soundness.* The native contract checks are conservative (they flag only definite faults); typestate and typed-contract precision depend on annotation quality, which is why STEP CHECK reports inferred findings as warnings.

7 Related Work

Serverless workflow orchestration. The commercial orchestrators—AWS Step Functions, Azure Durable Functions, and IBM Composer—and their trade-offs have been analysed extensively [13,23,16], and a range of programming models has been proposed to make orchestration more expressive or data-aware [6,3,24,17]. These works target execution—performance, scalability, programming convenience. We are complementary: STEP CHECK targets *correctness*, statically, on the most widely deployed of these platforms, without changing how workflows run.

Durable execution and transactions. Durable-execution engines such as Temporal and Cadence [21,22], and systems like Beldi [25], provide exactly-once execution and transactional or Saga-like semantics *at run time*, via logs and recovery protocols. STEP CHECK instead establishes retry-safety and compensation properties *before* deployment and on the existing platform, adding no runtime machinery. The two are composable: a statically verified workflow can still run on a durable backend.

Typestate and behavioural types. Typestate originates with Strom and Yemini [20] and was extended to objects and API protocols [11]; session and multiparty session types give a rich theory of protocol conformance [15,12]. STEP CHECK adapts the core idea—only-in-a-valid-state composition—to serverless workflow tasks, trading the full generality of these systems for a decidable, linear-time check that integrates into an industrial deployment toolchain.

Saga transactions and compensation. The Saga pattern [14] underpins recovery in cloud-native systems [18], but compensation is typically expressed in code and validated at run time [10,8]. STEP CHECK contributes a *static* compensation-completeness check: every persistent operation must have a reachable compensator before the workflow ships.

Workflow and service-composition verification. A large body of work verifies workflow soundness with Petri nets [1,5] and business-process compositions with model checking [4]. These provide strong, general guarantees but require dedicated formalisms and are rarely wired into everyday cloud development. STEP CHECK deliberately targets a deployed industrial language with lightweight, linear checks and direct compilation.

Table 6. STEP CHECK vs. representative approaches. ✓ full, ◦ partial / runtime-only, – absent.

Approach	Contract	Typestate	Retry	Compensation	Static	AWS target
AWS Step Functions [2]	–	–	–	–	–	✓
CNCF Serverless Workflow [9]	–	–	◦	◦	–	◦
Temporal / Cadence [21,22]	–	–	◦	◦	–	–
Beldi [25]	–	–	◦	◦	–	◦
Petri-net / BPEL verification [5,4]	–	◦	–	–	✓	–
Session / typestate types [15,20]	◦	✓	–	–	✓	–
StepCheck	✓	✓	✓	✓	✓	✓

Data contracts and interface compatibility. Interface compatibility is usually enforced at test time via consumer-driven contracts, or statically for service interfaces and REST clients [19,7]. Our contract pass brings this producer/consumer reasoning to the data flow *inside* a workflow, both natively (JSONPath bindings) and, with schemas, as a typed field-subset check.

Summary. Table 6 positions STEP CHECK. To our knowledge it is the first system to combine typestate conformance, retry safety, compensation completeness, and data-contract checking into a single *static* verifier that compiles directly to executable AWS Step Functions.

8 Conclusion

Serverless workflow orchestration has become a backbone of cloud-native systems, yet its workflows are untyped configuration whose correctness is checked only when they run. We presented STEP CHECK, a static verifier that catches four classes of orchestration defect —contract mismatches, out-of-order business transitions, unsafe retries, and missing compensations— *before* deployment, and compiles verified workflows back to executable Amazon States Language. The key to analysing the workflows that already exist is a two-layer model: sound native checks that need nothing beyond ASL, paired with a lightweight annotation layer whose defaults are inferred from the naming and resource conventions that pervade real workflows.

Our evaluation on 193 real Step Functions workflows supports the three research questions. STEP CHECK detects the targeted defect classes (100% recall over 429 injected defects) and surfaces genuine issues in unmodified workflows (RQ1); it recovers idempotency and persistence at zero authoring cost with 83–91% accuracy and visible, correctable residual error (RQ2); and it verifies a workflow in 19 μ s on average while adding zero runtime overhead, with verified workflows running unmodified on real AWS Step Functions (RQ3).

The broader message mirrors the history of type systems: just as static typing moved classes of error from run time to compile time, lightweight static verification can do the same for serverless orchestration, on the industrial platforms developers already use. Several directions remain. We have already shown the architecture

generalizes by adding a CNCF Serverless Workflow frontend with no change to the analyses; further frontends (SAM, CDK) would widen reach; richer effect and ownership information would sharpen the contract and compensation analyses; and a precise data-flow analysis of `Map` and `Parallel` item contexts would extend native contract checking. We release STEPCheck, the corpus, and the evaluation harness to support replication and reuse.

References

1. van der Aalst, W.M.P.: The application of Petri nets to workflow management. *Journal of Circuits, Systems and Computers* **8**(1), 21–66 (1998). <https://doi.org/10.1142/S0218126698000043>
2. Amazon Web Services: AWS step functions developer guide (amazon states language). <https://docs.aws.amazon.com/step-functions/> (2026), accessed 2026-06-17
3. Arjona, A., García López, P., Sampé, J., Slominski, A., Villard, L.: Triggerflow: Trigger-based orchestration of serverless workflows. *Future Generation Computer Systems* (2021), arXiv:2106.00583
4. Bianculli, D., Ghezzi, C., Spoletini, P.: A model checking approach to verify BPEL4WS workflows. In: *IEEE Int. Conf. on Service-Oriented Computing and Applications (SOCA)*. pp. 13–20 (2007). <https://doi.org/10.1109/SOCA.2007.5>
5. Blondin, M., Mazowiecki, F., Offtermatt, P.: Verifying generalised and structural soundness of workflow nets via relaxations. In: *Int. Conf. on Computer Aided Verification (CAV)*. pp. 435–458 (2022). https://doi.org/10.1007/978-3-031-13188-2_23
6. Burckhardt, S., Gillum, C., Justo, D., Kallas, K., McMahon, C., Meiklejohn, C.S.: Durable functions: Semantics for stateful serverless. *Proc. ACM Program. Lang.* **5**(OOPSLA) (2021). <https://doi.org/10.1145/3485510>
7. Burnay, N., Lopes, A., Vasconcelos, V.T.: SafeRESTScript: Statically checking REST API consumers. *arXiv preprint* (2020), arXiv:2007.08048
8. Chang, E.Y., Geng, L.: SagaLLM: Context management, validation, and transaction guarantees for multi-agent LLM planning. *arXiv preprint* (2025), arXiv:2503.11951
9. Cloud Native Computing Foundation: Serverless workflow specification. <https://serverlessworkflow.io/> (2026), accessed 2026-06-17
10. Colombo, C., Pace, G.J.: Comprehensive monitor-oriented compensation programming. *arXiv preprint* (2014), arXiv:1404.0849
11. DeLine, R., Fähndrich, M.: Typestates for objects. In: *European Conf. on Object-Oriented Programming (ECOOP)*. pp. 465–490 (2004). https://doi.org/10.1007/978-3-540-24851-4_21
12. Deniérou, P.M., Yoshida, N., Bejleri, A., Hu, R.: Parameterised multiparty session types. *Logical Methods in Computer Science* (2012), arXiv:1208.6483
13. García López, P., Sánchez-Artigas, M., París, G., Barcelona Pons, D., Ruiz Ollobarren, Á., Arroyo Pinto, D.: Comparison of FaaS orchestration systems. In: *IEEE/ACM Int. Conf. on Utility and Cloud Computing Companion (UCC)* (2018), arXiv:1807.11248
14. Garcia-Molina, H., Salem, K.: Sagas. In: *ACM SIGMOD Int. Conf. on Management of Data*. pp. 249–259 (1987). <https://doi.org/10.1145/38713.38742>
15. Honda, K., Yoshida, N., Carbone, M.: Multiparty asynchronous session types. In: *ACM SIGPLAN-SIGACT Symp. on Principles of Programming Languages (POPL)*. pp. 273–284 (2008). <https://doi.org/10.1145/1328438.1328472>

16. Kulkarni, V., Reddy, N., Khare, T., Prasad, A.S., Babu, C., Simmhan, Y.: Characterizing FaaS workflows on public clouds: The good, the bad and the ugly. arXiv preprint (2025), arXiv:2509.23013
17. Li, Z., Xu, C., Chen, Q., Zhao, J., Chen, C., Guo, M.: DataFlower: Exploiting the data-flow paradigm for serverless workflow orchestration. In: Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS) (2023), arXiv:2304.14629
18. Mohammad, M.: Resilient microservices: A systematic review of recovery patterns, strategies, and evaluation frameworks. arXiv preprint (2025), arXiv:2512.16959
19. Seco, J.C., Ferreira, P., Lourenço, H., Ferreira, C., Ferrão, L.: Robust contract evolution in a typesafe microservices architecture. *The Art, Science, and Engineering of Programming* 4(3) (2020). <https://doi.org/10.22152/programming-journal.org/2020/4/10>
20. Strom, R.E., Yemini, S.: Tpestate: A programming language concept for enhancing software reliability. *IEEE Transactions on Software Engineering* **SE-12**(1), 157–171 (1986). <https://doi.org/10.1109/TSE.1986.6312929>
21. Temporal Technologies: Temporal: Durable execution and workflow orchestration. <https://temporal.io/> (2026), accessed 2026-06-17
22. Uber Technologies: Cadence: Fault-oblivious stateful workflow orchestration. <https://cadenceworkflow.io/> (2026), accessed 2026-06-17
23. Wen, J., Liu, Y.: An empirical study on serverless workflow service. arXiv preprint (2021), arXiv:2101.03513
24. Yu, M., Cao, T., Wang, W., Chen, R.: Following the data, not the function: Rethinking function orchestration in serverless computing. In: *USENIX Symp. on Networked Systems Design and Implementation (NSDI)* (2023), arXiv:2109.13492
25. Zhang, H., Cardoza, A., Chen, P.B., Angel, S., Liu, V.: Fault-tolerant and transactional stateful serverless workflows. In: *USENIX Symp. on Operating Systems Design and Implementation (OSDI)* (2020), arXiv:2010.06706